

Relazione tecnica – CyberCuisine

Paragrafo 0 — Struttura della root del progetto

Prima di entrare nel merito del codice vero e proprio, è utile orientarsi nella struttura della directory radice del progetto, perché ogni file e cartella che si trova a questo livello ha una funzione precisa nell'ecosistema di sviluppo moderno.

Cartelle principali

`src/` è il cuore del progetto: contiene tutto il codice sorgente dell'applicazione, suddiviso in sottocartelle tematiche (`html/` , `js/` , `css/` , `assets/`). È l'unica cartella che viene servita dal browser. Ogni file qui dentro è scritto a mano, senza step di compilazione o transpiling.

`docs/` raccoglie la documentazione del progetto. Al suo interno si trovano:

- `core_docs/` — i documenti più importanti: questa relazione tecnica (`relazione.md`) e il PDF della specifica originale del docente (`PWM_ProgettoAnnoAccademico20242025.pdf`). Qui può inoltre risiedere `possibili_domande.md` , materiale **personale** di preparazione alla discussione orale, escluso dal versionamento tramite `.gitignore` ;
- `screenshots/` — le schermate dimostrative che costituiscono le "prove di funzionamento" richieste dalla specifica.

File di configurazione a livello di root

`.editorconfig` è un file di configurazione universale che istruisce qualsiasi editor di codice (VS Code, IntelliJ, Vim, ecc.) a utilizzare le stesse convenzioni di formattazione: indentazione con 2 spazi, encoding UTF-8, fine riga Unix (`\n`), newline finale obbligatoria, pulizia automatica degli spazi bianchi a fine riga. Senza di esso, due sviluppatori che lavorano con editor diversi potrebbero introdurre diff inutili (es. CRLF vs LF su Windows/Mac) ad ogni commit. Grazie a `.editorconfig` , la formattazione è uniforme indipendentemente dall'ambiente.

`.prettierrc` e `.prettierignore` riguardano [Prettier](#), il formatter automatico di codice più diffuso nell'ecosistema JavaScript. `.prettierrc` definisce le regole di stile adottate nel progetto: virgolette doppie, punto e virgola obbligatorio, larghezza massima della riga a 100 caratteri, virgola finale omessa, fine riga Unix. `.prettierignore` elenca invece i percorsi che Prettier deve ignorare durante la formattazione automatica (`node_modules` , `.vscode` , file SVG, cartella `dist`). Insieme, questi due file garantiscono che chiunque esegua `prettier --write` ottenga sempre lo stesso risultato, eliminando le discussioni di stile durante le code review.

`.gitignore` è il file che dice a Git quali file e cartelle non devono mai essere versionati nel repository. In questo progetto sono esclusi: la cartella `node_modules/` (le dipendenze di sviluppo si reinstallano con un comando), i file di log, le cache dei tool di build, le variabili d'ambiente (`.env`), note personali come credenziali di test, e altri artefatti temporanei. Senza `.gitignore` , il repository si riempirebbe di migliaia di file irrilevanti che appesantiscono la cronologia e possono esporre informazioni sensibili.

`LICENSE` contiene la licenza del software. In questo progetto è la licenza **MIT**, che è la più permissiva e diffusa nel mondo open source: chiunque può usare, modificare e redistribuire il codice, anche a scopo commerciale, a patto di mantenere il testo della licenza originale. Includere una licenza è considerata buona pratica anche per i progetti accademici, perché rende espliciti i termini d'uso del codice.

`README.md` è il documento di benvenuto del repository: spiega in breve cos'è il progetto, come avviarlo, le dipendenze esterne e le note di manutenzione essenziali. È la prima cosa che si legge aprendo il repository su GitHub. Contiene anche le istruzioni per il reset del localStorage durante le sessioni di test.

Perché `.md` e non `.txt`

Il formato **Markdown** (`.md`) è oggi lo standard de facto per la documentazione tecnica nei progetti software, mentre i file `.txt` sono considerati un retaggio dello sviluppo anni '90. I motivi sono pratici: Markdown viene renderizzato automaticamente da GitHub, VS Code, Obsidian e qualsiasi moderna piattaforma di sviluppo, producendo titoli, grassetti, elenchi, blocchi di codice con syntax highlighting e link cliccabili. Lo stesso contenuto in `.txt` è solo testo piatto, non navigabile e non leggibile a colpo d'occhio. Markdown mantiene inoltre la leggibilità anche in forma grezza (a differenza di HTML o Word), il che lo rende perfetto per la versionizzazione con Git: le diff sono chiare e significative. Per questi motivi tutti i documenti di questo progetto sono in formato `.md`.

1. Introduzione e obiettivo del progetto

CyberCuisine è un'applicazione web molto minimale sviluppata come progetto d'esame per il corso di **Programmazione Web e Mobile** dell'Università degli Studi di Milano (A.A. 2024/2025). L'obiettivo è realizzare una **Piattaforma per la Gestione di Ricette di Cucina (PGRC)**: un portale web dove un utente registrato può esplorare un vasto catalogo di ricette culinarie, costruire il proprio ricettario personale, aggiungere note private e lasciare recensioni strutturate sui piatti provati.

Il progetto è stato sviluppato interamente in **tecnologie web standard** — HTML5, CSS3 e JavaScript vanilla — senza l'ausilio di framework JS (niente React, Vue o Angular), senza

backend e senza database server-side. Tutta la persistenza dei dati avviene nel **web storage del browser** (localStorage e sessionStorage), in formato JSON. I dati delle ricette vengono recuperati in tempo reale dalle **API REST pubbliche di TheMealDB** e successivamente memorizzati in locale per garantire navigazione veloce e funzionamento parziale anche offline.

La specifica del docente richiedeva esplicitamente che all'avvio dell'applicazione tutti i dati necessari venissero scaricati dalle API, memorizzati nel web storage e visualizzati — requisito che ha guidato l'intera strategia di startup e caching descritta nelle sezioni successive.

2. Requisiti del docente e scelte implementative

La specifica individua quattro macro-scenari principali, ciascuno con i propri requisiti funzionali. Di seguito si riportano i requisiti e le scelte adottate per soddisfarli.

2.1 Gestione del profilo utente

Il primo macro-scenario richiede registrazione, modifica dei dati personali e rimozione del profilo. La registrazione raccoglie: nome, cognome, username, email, password (con conferma), paese di origine, paese di residenza e piatti preferiti (campo libero separato da virgole). La password non viene mai salvata in chiaro: viene generato un **salt** casuale (16 byte via `crypto.getRandomValues`) e calcolato l'hash **SHA-256** della stringa `salt:password` tramite la Web Crypto API nativa del browser. Solo `passwordHash` e `salt` vengono scritti nel localStorage; la password in chiaro rimane esclusivamente in una variabile locale durante l'elaborazione e non viene mai serializzata. La modifica del profilo richiede una ri-autenticazione tramite modale (inserire la password corrente prima di sbloccare i campi), il che difende da modifiche accidentali o non autorizzate in sessioni lasciate aperte. L'eliminazione del profilo rimuove in cascata l'utente, il suo ricettario e tutte le sue recensioni.

2.2 Ricerca di ricette culinarie

Il secondo macro-scenario richiede la ricerca per nome del piatto, ingredienti principali e lettera iniziale, oltre alla "ricerca sequenziale" (sfogliare l'intero catalogo). Poiché all'avvio il catalogo completo (alcune centinaia di ricette, ~300 alla data di sviluppo) viene scaricato e salvato in localStorage, **tutte le ricerche avvengono in locale**, senza ulteriori chiamate di rete: il risultato è istantaneo e funziona anche offline (dopo il primo caricamento). La ricerca per nome usa un `match includes case-insensitive`; quella per ingrediente scorre la lista dei 20 ingredienti normalizzati di ogni ricetta; quella per lettera confronta il primo carattere del nome. La modalità "Sfoglia tutto" presenta l'intero catalogo raggruppato per lettera con un indice alfabetico sticky che si aggiorna durante lo scroll grazie a un `IntersectionObserver`.

2.3 Gestione del ricettario personale

Il terzo macro-scenario richiede la possibilità di aggiungere e rimuovere ricette dal ricettario personale (creato automaticamente alla registrazione) e di associare a ciascuna una nota testuale privata. Il ricettario è salvato in localStorage sotto la chiave `cookbook:<idUtente>` come oggetto `{ recipeIds: [...], notesByRecipeId: { idRicetta: nota } }`. Si memorizzano solo gli ID delle ricette (non l'intera scheda), perché i dettagli sono già nella cache del catalogo e possono essere recuperati in $O(1)$. Le note sono visibili solo all'utente proprietario e non compaiono nelle recensioni pubbliche della ricetta. Aggiunta e rimozione passano entrambe attraverso una **modale di conferma** (componente riutilizzabile), evitando rimozioni accidentali.

2.4 Recensioni delle ricette

L'ultimo macro-scenario richiede che ogni utente possa recensire qualsiasi ricetta specificando: data di preparazione, voto da 1 a 5 per la difficoltà e voto da 1 a 5 per il gusto.

L'implementazione usa un pattern **upsert**: un utente ha al massimo una recensione per ricetta; se esiste già, il salvataggio la aggiorna anziché crearne una duplicata. Le recensioni sono raggruppate per ricetta nella chiave `reviews:<idRicetta>` e sono visibili a tutti gli utenti nella scheda della ricetta. Solo il proprietario vede il pulsante di rimozione. L'inserimento e la modifica avvengono tramite la stessa modale (precompilata con i valori esistenti se già recensita), la rimozione richiede conferma esplicita.

2.5 Matrice di conformità ai requisiti

La tabella seguente mostra in modo immediato la corrispondenza tra ogni requisito della specifica del docente e la sua implementazione nell'applicazione.

Requisito (specifica)	Stato	Dove / Note
Registrazione (username, email, password, piatti preferiti)	✓	<code>vista-registrazione.js</code> + <code>register.html</code> ; scrive in <code>users</code>
Login	✓	<code>vista-accesso.js</code> ; verifica via hash; scrive <code>session</code>
Logout	✓	<code>navbar.js</code> → <code>gestisciLogout</code> ; azzera <code>session</code>
Modifica dati personali	✓	<code>vista-profilo.js</code> ; re-auth con password prima della modifica
Rimozione profilo	✓	<code>rimuoviUtente</code> (cancella utente + ricettario + recensioni)
Ricerca per nome	✓	<code>cercaRicettePerNome</code> (filtro sulla cache locale)
Ricerca per ingrediente	✓	<code>cercaRicettePerIngrediente</code>

Requisito (specifica)	Stato	Dove / Note
Ricerca per lettera iniziale	✓	<code>cercaRicettePerLettera</code>
Ricerca sequenziale (sfoglia tutto)	✓	"Sfoglia l'intero catalogo" in Esplora, indice A–Z
Scheda ricetta: ingredienti, immagini, procedimento	✓	<code>vista-dettaglio-ricetta.js</code>
Scheda ricetta: mostra recensioni utenti	✓	elenco recensioni da <code>reviews:<idRicetta></code>
Ricettario personale creato alla registrazione	✓	inizialmente vuoto (<code>cookbook:<idUtente></code>)
Aggiungi/Rimuovi ricetta dal ricettario	✓	<code>aggiornaRicettario</code> + pulsanti su card e dettaglio
Nota privata per ricetta (non visibile ad altri)	✓	<code>notesByRecipeId</code> nel ricettario
Recensione: data preparazione + difficoltà 1–5 + gusto 1–5	✓	<code>modale-recensione.js</code> (<code>cookedAt</code> , <code>difficulty</code> , <code>taste</code>)
Inserimento recensioni	✓	salvataggio in <code>reviews:<idRicetta></code> (upsert per utente)
Rimozione recensioni	✓	<code>rimuoviRecensione</code> + pulsante in scheda e in "Le tue recensioni"
Startup: dati scaricati da TheMealDB → web storage → visualizzati	✓	<code>precaricaCatalogoCompleto()</code> in <code>main.js</code> con overlay
Dati in web storage in formato JSON	✓	tutte le chiavi serializzate con <code>JSON.stringify/parse</code>
Separazione struttura HTML5 / presentazione CSS3	✓	nessuno stile inline; tutto in <code>src/css/*</code>
Implementazione in HTML5 + CSS3 + JavaScript	✓	nessun framework JS, nessun build step

3. Architettura dell'applicazione

3.1 Single Page Application con hash routing

CyberCuisine è strutturata come **SPA (Single Page Application)**: il browser carica una sola pagina HTML (`src/html/index.html`) e tutte le navigazioni successive avvengono senza reload completo, manipolando dinamicamente il contenuto del `<main id="app">` . Questo

approccio garantisce transizioni fluide, nessuna perdita dello stato in-memory tra le viste e un'esperienza più simile a un'applicazione desktop che a un sito tradizionale.

La navigazione si basa sull'**hash routing**: ogni vista corrisponde a un hash nell'URL (es. `#/home`, `#/esplora`, `#/ricetta/52772`). Quando l'hash cambia, il browser emette un evento `hashchange` che il router intercetta senza alcun reload di pagina. Questo meccanismo funziona anche senza un server web configurato ad hoc (nessun problema di 404 su refresh), il che lo rende ideale per un progetto statico aperto con Live Server.

3.2 Il router (`router.js` e `rotte.js`)

Il router è il componente centrale dell'architettura. Al cambio di hash, `gestisciCambioRoute` esegue questa sequenza:

1. Normalizza l'hash (se mancante o non valido, usa `#/home` come default).
2. Estrae eventuali parametri dinamici (es. l'id in `#/ricetta/52772`).
3. Cerca la configurazione della rotta nella mappa esportata da `rotte.js`.
4. Controlla se la rotta è protetta e se esiste una sessione attiva.
5. Sceglie il frammento HTML corretto (versione pubblica o autenticata per le rotte con doppia variante, come la home).
6. Carica il frammento via `fetch`, con caching in-memory in `statoApp.cacheFrammenti` per evitare richieste ripetute.
7. Inietta l'HTML nel contenitore `#app`.
8. Richiama la funzione di inizializzazione della vista (`alCaricamento` o `alCaricamentoProtetto`).
9. Aggiorna l'indicatore del link attivo nella navbar e lo stato dell'area autenticazione.

`rotte.js` è puramente dichiarativo: associa ogni hash a un oggetto che specifica il file HTML da caricare, la funzione JS da eseguire e se la rotta richiede autenticazione. Aggiungere una nuova vista all'applicazione richiede solo tre passi: creare il frammento HTML, scrivere il file `vista-*.js`, e aggiungere una riga in `rotte.js`.

3.3 Il punto di ingresso (`main.js`)

`main.js` è l'unico file caricato da `index.html` come modulo ES6. La sua funzione `inizializzaApp` segue un ordine preciso e obbligatorio:

1. **Inizializza/migra il web storage** (`inizializzaStorage`): crea le chiavi base se mancano, o avvia la migrazione se lo schema è di una versione precedente.
2. **Aggancia la navbar** (`impostaEventiAuthNav`): collega il pulsante logout e sincronizza l'aspetto della navbar con la sessione corrente.

3. **Registra la event delegation** (`impostaAzioniCarteRicetta`): un solo listener sul `document` gestisce tutti i pulsanti delle card (ricettario, recensioni, dettaglio) anche per elementi creati dopo il caricamento iniziale.
 4. **Scarica il catalogo completo** (`precaricaCatalogoCompleto`): è la fase più lunga al primo avvio; durante questa operazione viene mostrato l'overlay di caricamento.
 5. **Attiva il router** e renderizza la vista iniziale.
-

4. Modello dati e web storage

4.1 Schema versionato

Il web storage è organizzato secondo uno **schema versionato** (attualmente `SCHEMA_VERSION = 2`). La versione è registrata nella chiave `app:meta` insieme al timestamp dell'ultima inizializzazione e ai metadati della cache API. Ogni volta che l'applicazione si avvia, controlla se la versione dello schema corrisponde a quella attesa; se non corrisponde, esegue una migrazione automatica e non distruttiva che converte i dati vecchi nel formato nuovo.

4.2 Chiavi del localStorage

Chiave	Forma	Contenuto
<code>app:meta</code>	oggetto	Versione schema, timestamp inizializzazione, info cache API
<code>users</code>	array	Tutti gli account registrati
<code>session</code>	oggetto	<code>{ currentUserId, loginAt }</code> — chi è loggato adesso
<code>recipes:cache</code>	oggetto	<code>{ updatedAt, byId }</code> — catalogo TheMealDB completo
<code>areas:cache</code>	oggetto	<code>{ updatedAt, items, derivedFromCatalog }</code> — cucine disponibili
<code>cookbook:<idUtente></code>	oggetto	<code>{ recipeIds, notesByRecipeId }</code> — ricettario personale
<code>reviews:<idRicetta></code>	array	Recensioni per quella ricetta
<code>cc_remembered_accounts</code>	array	Account del "Ricordami": <code>{ username, displayName, encryptedPassword, iv, lastUsed }</code> con password cifrata AES-GCM (vedi §7.2)

Ogni utente nel array `users` ha la forma:

```
{
  "id": "utente_1748000000_123",
  "username": "mariorossi",
  "email": "mario@email.it",
  "passwordHash": "VDbVAwVT+e80/...",
  "salt": "zdYf1qAUJPD8h...",
  "firstName": "Mario",
  "lastName": "Rossi",
  "originCountry": "Italian",
  "residenceCountry": "Italian",
  "favoriteDishes": ["Lasagna", "Risotto"],
  "createdAt": "2026-06-04T10:00:00.000Z",
  "updatedAt": "2026-06-04T10:00:00.000Z"
}
```

Ogni recensione nell'array `reviews:<idRicetta>` ha la forma:

```
{
  "id": "recensione_1748000000_456",
  "userId": "utente_1748000000_123",
  "cookedAt": "2026-06-01",
  "difficulty": 3,
  "taste": 5,
  "commento": "Ottima, l'ho rifatta tre volte.",
  "createdAt": "2026-06-04T10:05:00.000Z",
  "updatedAt": "2026-06-04T10:05:00.000Z"
}
```

4.3 Chiavi del sessionStorage

Il sessionStorage ospita **una sola** chiave, legata al passaggio di consegne tra registrazione e login:

- `cc_post_signup`: messaggio one-shot che la registrazione scrive prima di reindirizzare al login. La pagina di accesso lo legge, lo mostra all'utente ("Account creato, accedi per continuare") e lo cancella immediatamente. Questo evita di dover passare dati tra viste tramite parametri URL o stato globale.

Il "Ricordami" **non** usa il sessionStorage: è stato implementato in

`localStorage["cc_remembered_accounts"]` con password cifrata AES-GCM (vedi §7.2), così gli account ricordati sopravvivono anche alla chiusura del browser. Una precedente versione dell'app usava a questo scopo una chiave `cc_accesso_ricorda` in sessionStorage, oggi completamente rimossa dal codice.

4.4 Separazione sessione e dati utente

Una scelta implementativa deliberata è la separazione netta tra la chiave `users` (che contiene i dati di tutti gli account registrati) e la chiave `session` (che contiene solo l'id dell'utente loggato in questo momento). Il logout azzerava solo `session.currentUserId`, senza toccare `users`: l'account esiste ancora, le ricette salvate e le recensioni sono intatte, e al successivo login tutto torna disponibile. Questa separazione rende anche più semplice individuare "chi è loggato" senza scorrere un array.

5. Pipeline di startup e gestione del catalogo

5.1 Il problema dello startup

La specifica richiede che all'avvio tutti i dati necessari vengano scaricati dalle API di TheMealDB, memorizzati nel web storage e visualizzati. Questo crea una tensione: scaricare l'intero catalogo (alcune centinaia di ricette, ~300 alla data di sviluppo, tramite 26 chiamate, una per lettera) può richiedere diversi secondi su connessioni lente. La soluzione adottata bilancia conformità al requisito e praticità d'uso.

5.2 Flusso di primo avvio

1. `inizializzaStorage` crea le chiavi base nel `localStorage` (o migra quelle legacy).
2. `precaricaCatalogoCompleto` controlla se `recipes:cache` è già popolata e valida (TTL non scaduto). Al primo avvio la cache è vuota, quindi si procede con il download.
3. Viene mostrato l'**overlay di caricamento** (`#ccAvvioOverlay`, visibile di default nell'HTML e nascosto via JS al termine), così l'utente vede un feedback visivo invece di una pagina bianca.
4. Vengono eseguite 26 fetch in parallelo (`Promise.all`) verso `search.php?f=<lettera>`, una per ogni lettera dell'alfabeto. Le risposte vengono appiattite, normalizzate e indicizzate per id (eliminando automaticamente i duplicati).
5. Il dizionario risultante viene scritto in `recipes:cache.byId` con il timestamp `updatedAt` e i metadati `strategy: "full", complete: true`.
6. L'overlay viene rimosso, il router renderizza la vista iniziale.

5.3 Riavvii successivi (cache hit)

Ai riavvii successivi, se la cache è presente e il TTL di 72 ore non è scaduto, `precaricaCatalogoCompleto` restituisce immediatamente i dati già in `localStorage` senza alcuna chiamata di rete. L'overlay sparisce quasi istantaneamente. Allo scadere del TTL, il prossimo avvio ricarica il catalogo aggiornato.

5.4 Gestione delle aree/paesi

L'endpoint `list.php?a=list` di TheMealDB restituisce ~195 demonimi (Afghan, Caymanian, ecc.) la maggior parte dei quali non ha ricette nel catalogo. Usare quell'endpoint per popolare i menu "Paese di origine/residenza" avrebbe prodotto un elenco di 195 voci quasi tutte inutili. La scelta adottata è stata di **derivare le aree disponibili dalle ricette effettivamente presenti nel catalogo**: si estraggono le ~37 aree distinte (`areaCodice`) dalle ricette scaricate, le si arricchisce con nome italiano e emoji bandiera, e le si ordina alfabeticamente. Ogni paese selezionabile produce quindi ricette reali nella home personalizzata.

Le bandiere emoji non sono visibili su Windows senza un intervento esplicito, perché il font di sistema (Segoe UI Emoji) disegna i caratteri *regional indicator* come sigle bilettera invece che come bandiere. Per risolvere, è stato integrato il webfont self-hosted **Twemoji Country Flags** (`src/assets/fonts/TwemojiCountryFlags.woff2`), dichiarato in `base.css` con un `@font-face` che limita il suo intervento al solo intervallo Unicode dei regional indicator (`U+1F1E6-U+1F1FF`), senza alterare alcun altro testo.

6. Flussi funzionali dettagliati

6.1 Registrazione

1. L'utente compila il form (`register.html`) con tutti i campi obbligatori e i piatti preferiti.
2. Il client valida: campi vuoti, password di almeno 6 caratteri, corrispondenza tra password e conferma, unicità di username ed email.
3. Prima dell'accettazione, l'utente deve leggere i documenti legali (Termini e condizioni, Informativa privacy) in due modali dedicate: il pulsante di chiusura ("Ho letto, torna al form") è disabilitato finché non si raggiunge il fondo del documento via scroll.
4. Si genera un `salt` casuale (16 byte) e si calcola `passwordHash = SHA-256(salt:password)` via Web Crypto API.
5. Il nuovo utente viene aggiunto all'array `users` nel `localStorage`. La password in chiaro non viene mai scritta da nessuna parte.
6. Un messaggio one-shot viene scritto in `sessionStorage` (`cc_post_signup`) e l'utente viene rediretto alla pagina di login.

6.2 Login

1. L'utente inserisce username/email e password.
2. Il client cerca l'utente nell'array `users` per username, nomeUtente o email.
3. Viene ricalcolato `SHA-256(salt:passwordInserita)` usando il salt dell'utente trovato e confrontato con il `passwordHash` memorizzato.

4. Se corrispondono, la sessione viene aperta scrivendo `{ currentUserId: utente.id, loginAt: now }` nella chiave `session`.
5. Se è spuntato "Ricordami", viene salvato solo l'identificatore (mai la password) in `sessionStorage`.
6. La navbar si aggiorna, mostrando il menu "Ciao <nome>" al posto del link di accesso.

6.3 Refresh della pagina e persistenza della sessione

Il refresh non provoca il logout: poiché `session` è in `localStorage` (persistente tra reload e riaperture del browser), `ottieniUtenteCorrente` la ritrova al prossimo avvio e l'utente rimane loggato. La funzione `inizializzaStorage` è idempotente: crea le chiavi mancanti ma non sovrascrive quelle esistenti, quindi la sessione sopravvive intatta.

6.4 Rotte protette

Alcune rotte (Profilo, Ricettario, Recensioni) richiedono autenticazione. Il router chiama `ottieniUtenteCorrente` prima di renderizzare: se la funzione restituisce `null` (chiave `session` vuota o utente non trovato), l'utente viene rediretto a `#/accesso`. Questo controllo avviene a ogni cambio di rotta, rendendo impossibile accedere alle sezioni protette svuotando la sessione dal DevTools.

6.5 Logout

Il logout scrive `{ currentUserId: null, loginAt: null }` nella chiave `session`. L'array `users`, il ricettario e le recensioni restano intatti: il logout chiude solo la sessione di lavoro corrente, non elimina l'account.

6.6 Ricerca ricette

L'utente può cercare in tre modalità dalla vista Esplora:

- **Per nome:** `match includes case-insensitive` su `ricetta.nome`.
- **Per ingrediente:** almeno un ingrediente della ricetta contiene il termine cercato.
- **Per lettera iniziale:** il primo carattere del nome corrisponde alla lettera digitata.

Tutte e tre le modalità filtrano la cache locale, senza chiamate di rete. Il risultato viene visualizzato in una griglia responsiva di card; nella modalità "Sfoglia tutto" le ricette sono raggruppate per lettera con un indice alfabetico sticky che usa un `IntersectionObserver` per aggiornare la lettera evidenziata durante lo scroll.

6.7 Ricettario personale

Il ricettario mostra le ricette salvate dall'utente. Per ciascuna viene recuperata la scheda completa (dalla cache o via `lookup.php?i=<id>` come fallback) e renderizzata una card con

immagine, categoria, area, **campo nota privata**, pulsante di recensione e pulsante di rimozione. Sia l'aggiunta che la rimozione passano per una modale di conferma che mostra il nome della ricetta, evitando azioni accidentali.

La **nota privata** richiesta dalla specifica è gestita direttamente sulla card: una `textarea` precompilata con l'eventuale nota già salvata e un pulsante "Salva nota". Al salvataggio, `aggiornaNotaRicettario(idRicetta, testo)` scrive il testo in `cookbook`: `<idUser>.notesByRecipeId[idRicetta]`; un indicatore "Nota salvata ✓" conferma l'operazione. La nota è **privata** per costruzione: vive solo nel ricettario del singolo utente e non compare mai nelle recensioni pubbliche della ricetta. Il testo viene sottoposto a `escape` prima di essere reiniettato nella `textarea`, così una nota contenente caratteri come `< o </textarea>` non rompe il markup.

Il pulsante recensione è contestuale: mostra "Scrivi recensione" se l'utente non ha ancora recensito la ricetta, "Modifica recensione" se ne esiste già una — calcolato in tempo reale leggendo `reviews:*` dal `localStorage` ad ogni render.

6.8 Recensioni

Dalla scheda dettaglio di qualsiasi ricetta, l'utente loggato può aprire la modale di recensione. Se ha già recensito quella ricetta, la modale si apre precompilata con i valori esistenti (comportamento `upsert`). I campi richiesti dalla specifica sono: data di preparazione (`<input type="date">`), difficoltà 1–5 (select), gusto 1–5 (select); il commento testuale è opzionale. Dopo il salvataggio la vista si aggiorna immediatamente senza reload (re-render live): il pulsante passa da "Scrivi recensione" a "Modifica recensione" all'istante.

Nella scheda ricetta l'elenco delle recensioni è ottimizzato per il contesto: il nome del piatto e il pulsante "Vai alla ricetta" sono omessi (ridondanti, si è già nella sua pagina); la recensione dell'utente loggato appare sempre per prima. La rimozione apre la modale di conferma custom (non `window.confirm`).

Dalla pagina "Le tue recensioni" (`#/recensioni`) l'utente vede tutte le proprie recensioni con accesso rapido alla ricetta corrispondente. Cliccando "Guarda la tua recensione" da qualsiasi card dell'app, l'app naviga a `#/recensioni`, fa autoscroll alla card corrispondente e la evidenzia con un glow intermittente ciano/viola per 3 secondi — così l'utente la identifica immediatamente anche in una lista lunga.

7. Sicurezza e gestione delle credenziali

7.1 Hashing delle password

La Web Crypto API (`crypto.subtle.digest`) è l'API crittografica nativa dei browser moderni, standardizzata da W3C. In questo progetto viene usata per calcolare hash SHA-256. La stringa da hashare è `salt:password`, dove il salt è una sequenza di 16 byte casuali generati da `crypto.getRandomValues` (sorgente crittograficamente sicura, a differenza di `Math.random`). Il risultato è un `ArrayBuffer` di 32 byte che viene codificato in Base64 per la serializzazione JSON.

L'uso del salt per ogni utente è fondamentale: anche se due utenti scegliessero la stessa password, i loro hash sarebbero completamente diversi. Questo rende inefficaci gli attacchi con **rainbow table** (tabelle precostruite di hash noti). In un contesto didattico senza backend, questo rappresenta il massimo livello di sicurezza raggiungibile lato client.

7.2 Cifratura AES-GCM per gli account ricordati

La funzione "Ricordami" utilizza un meccanismo più sofisticato del semplice salvataggio dell'identificatore: quando l'utente fa login con la casella spuntata, la password in chiaro viene **cifrata con AES-256-GCM** (Web Crypto API) prima di essere scritta in `localStorage["cc_remembered_accounts"]`. La chiave AES è derivata direttamente dal `passwordHash` SHA-256 dell'utente (già presente in `users`), che è di 32 byte — la dimensione esatta richiesta da AES-256. Per ogni salvataggio viene generato un IV casuale da 12 byte (standard AES-GCM).

In DevTools la voce `encryptedPassword` appare come un blob Base64 opaco, mai come testo leggibile. Al successivo accesso alla pagina di login, dopo 2 secondi appare un picker con gli account ricordati: selezionandone uno, l'app decifra la password on-the-fly (leggendo il `passwordHash` da `users`) e la inserisce con un effetto typewriter nel campo password. Se la decifratura fallisce (es. l'account è stato eliminato e ricreato, cambiando l'hash), il focus passa semplicemente al campo password per la digitazione manuale.

7.3 Limiti consapevoli

È importante essere onesti sui limiti di questo approccio: non esiste un backend che validi le richieste, quindi un utente tecnicamente capace potrebbe manipolare il `localStorage` direttamente. Tuttavia questo esula completamente dagli obiettivi didattici del corso, che richiede esplicitamente la persistenza su web storage senza backend. L'implementazione scelta è la più robusta possibile entro questi vincoli. Nota: la password cifrata in `cc_remembered_accounts` può essere decifrata da chi ha già accesso al `localStorage` (perché la chiave è il `passwordHash` presente nello stesso storage) — ma non è comunque mai in chiaro a un'ispezione diretta.

8. UI, UX e componenti riutilizzabili

8.1 Architettura CSS

Il CSS è organizzato in file tematici separati, caricati tutti da `index.html` :

- `theme.css` : variabili CSS globali (palette cyberpunk — viola `#b312ff` , ciano `#00f0ff` , sfondo quasi-nero `#0c0b1a`), che fungono da design token per tutto il resto.
- `base.css` : reset minimo, font-face per le bandiere emoji, font-family del body, stili base per titoli e link.
- `layout.css` : header sticky, navbar, footer, hero section, layout della vista Esplora.
- `components.css` : card con effetto glow, pulsanti, indice alfabetico, elementi del ricettario.
- `forms.css` : input, select, placeholder, testi di aiuto, modali documenti legali, spinner di caricamento.
- `modal.css` : stile delle modali Bootstrap personalizzate, pulsante di chiusura unificato.
- `utilities.css` : classi helper varie.

8.2 Pulsante di chiusura unificato

Tutti i pulsanti "×" dell'applicazione (modali Bootstrap, modali documenti, menu offcanvas mobile) condividono la stessa classe CSS e lo stesso aspetto: sfondo rosso `#c0392b` , bordo `#e74c3c` , dimensione `2×2rem`, bordi arrotondati, posizionamento `absolute` nell'angolo in alto a destra del contenitore. L'effetto hover aumenta leggermente il tono rosso e aggiunge un glow. Questa uniformità è un esempio concreto di componente riutilizzabile a livello di stile.

8.3 Event delegation

I pulsanti sulle card ricetta (aggiungi/rimuovi dal ricettario, scrivi/guarda recensione, apri dettaglio) non hanno listener diretti. Invece, un singolo listener è registrato sul `document` e intercetta i click su qualsiasi elemento con i `data-*` attributi rilevanti. Questo pattern è necessario perché le card vengono ricreate via `innerHTML` ad ogni navigazione: i listener diretti andrebbero persi, mentre la delegation sul documento è registrata una sola volta allo startup e funziona per qualsiasi elemento presente o futuro nel DOM.

8.4 Modali di conferma

Tutte le azioni potenzialmente distruttive (aggiunta/rimozione dal ricettario, logout, eliminazione profilo, rimozione recensione) passano per una modale di conferma prima di essere eseguite. La modale è definita una sola volta in `index.html` e viene riutilizzata per tutte le azioni tramite la funzione `mostraModalConfermaGenerica(titolo, messaggio, onConfirm)` esportata da `azioni-card.js` — eliminando completamente `window.confirm()` dall'intera applicazione (alert nativo del browser, non integrato nella UI). Aggiornandone titolo e messaggio di volta in volta, un singolo componente copre tutti i casi d'uso.

8.5 UX di validazione form

La validazione dei form segue pattern professionali consolidati (GitHub, Stripe, Airbnb):

- **Scroll automatico all'errore:** `mostraAvviso(elemento, messaggio, "danger")` fa automaticamente `scrollIntoView` sull'alert; l'utente non deve scorrere manualmente per trovare il messaggio di errore anche in form lunghi come la registrazione.
- **Glow rosso sui campi invalidi:** la funzione riutilizzabile `evidenziaFieldInvalidi(campi)` aggiunge la classe `.cc-campo-invalido` (animazione CSS con bordo e glow rosso) a ogni campo non compilato. La classe si auto-rimuove al primo `input` dell'utente su quel campo.
- **Fix autofill browser:** Chrome/Edge colorano i campi auto-completati con sfondo chiaro ignorando il CSS. Il fix industry-standard (`-webkit-box-shadow: inset + transition: background-color 5000s`) forza il tema scuro anche sui campi compilati automaticamente.
- **Checkbox legali bloccati:** i checkbox "Ho letto e accetto Termini" e "Ho letto e accetto Privacy" partono `disabled` e diventano abilitati (e auto-spuntati) solo dopo aver aperto la modale corrispondente e cliccato il pulsante "Ho letto, torna al form" — che a sua volta si sblocca solo dopo aver scrollato tutto il documento. Il bypass manuale è fisicamente impossibile.

8.5 Feedback di caricamento

L'applicazione fornisce feedback visivo in tutti i momenti di attesa:

- **Overlay di avvio:** copre l'intera pagina durante il download del catalogo iniziale.
- **Spinner inline** (piadina rotante): appare nelle viste Esplora, Login e Registrazione durante le operazioni asincrone.
- **Testi placeholder:** "Caricamento ricette..." nei contenitori prima che i dati arrivino.
- **Badge contatori:** aggiornati dinamicamente per mostrare quante ricette sono state trovate/caricate.

8.6 Accessibilità e standard web

Oltre all'estetica, l'applicazione adotta una serie di accorgimenti conformi alle linee guida di accessibilità (WCAG) e alle best practice del web standard:

- `prefers-reduced-motion` : una media query globale (`base.css`) azzera animazioni e transizioni per gli utenti che hanno attivato la riduzione del movimento nelle impostazioni di sistema (utile per chi soffre di disturbi vestibolari o emicrania). L'app resta pienamente funzionale, semplicemente "ferma".
- `aria-current="page"` : il link di navigazione attivo, oltre alla classe `.active` (puramente visiva), espone l'attributo ARIA che gli screen reader annunciano come "pagina corrente".

- `aria-live="polite"` : i badge che mostrano il conteggio delle ricette vengono annunciati dagli screen reader quando il loro valore cambia dinamicamente, senza interrompere la lettura.
- `autocomplete` : i campi di login e registrazione dichiarano i token standard (`username` , `current-password` , `new-password` , `email` , `given-name` , `family-name`), abilitando il corretto funzionamento dei gestori di password e migliorando l'esperienza di compilazione.
- `<noscript>` : essendo una SPA dipendente da JavaScript, un fallback avvisa esplicitamente l'utente qualora JavaScript fosse disabilitato, invece di mostrare una pagina vuota.
- `<head>` **completo**: `meta description` (anteprime/SEO), `theme-color` (chrome del browser mobile allineata al tema scuro) e `favicon` (icona del tab, oltre a evitare la richiesta 404 di `favicon.ico`).
- **HTML semantico** (`<header>` , `<nav>` , `<main>` , `<section>` , `<footer>`), label associate a ogni campo, `alt` su tutte le immagini e `lang="it"` sul documento completano l'impianto accessibile.

9. Struttura dei moduli JavaScript e dipendenze

9.1 Grafo delle dipendenze

```

main.js
├─ storage.js (inizializzaStorage)
├─ navbar.js (impostaEventiAuthNav)
├─ componenti/azioni-card.js (impostaAzioniCarteRicetta)
├─ gestione-api/api.js (precaricaCatalogoCompleto)
└─ router.js (gestisciCambioRoute)
    ├─ rotte.js → viste/vista-*.js
    ├─ stato.js
    ├─ navbar.js (aggiornaNavigazioneAttiva, aggiornaStatoAuthNav)
    └─ storage.js (ottieniUtenteCorrente)

viste/vista-*.js
├─ storage.js (lettura/scrittura dati dominio)
├─ gestione-api/api.js (ricerca, recupero ricette)
├─ componenti/carte.js (generatori HTML card)
└─ componenti/azioni-card.js, modale-recensione.js

gestione-api/api.js ↔ storage.js (cache reciproca)
auth.js ← storage.js, viste/vista-accesso.js, vista-registrazione.js, vista-
```



```
profilo.js
costanti.js ← tutti i moduli (chiavi storage, BASE_API)
```

9.2 Responsabilità di ogni modulo

- `costanti.js` : unica fonte di verità per le chiavi del web storage e l'URL base delle API. Importato da quasi tutti gli altri moduli.
- `auth.js` : generazione salt, hashing SHA-256, verifica password. Nessuna dipendenza da DOM o storage.
- `stato.js` : oggetto singleton con stato volatile in-memory (cache frammenti HTML, risultati ricerca, percorso attivo).
- `storage.js` : tutto ciò che riguarda la persistenza — lettura/scrittura localStorage, normalizzazione utenti, CRUD ricettario e recensioni, migrazione schema. Importa `auth.js` solo per la migrazione dei dati legacy.
- `gestione-api/api.js` : comunicazione con TheMealDB, normalizzazione dei dati raw in formato interno, gestione cache catalogo e aree.
- `navbar.js` : unico responsabile dell'aspetto della navbar in risposta allo stato di autenticazione.
- `router.js` : caricamento frammenti HTML e dispatch delle funzioni di inizializzazione delle viste.
- `rotte.js` : mappa dichiarativa hash → configurazione (frammento, handler, flag protetta).
- `componenti/carte.js` : generatori di markup HTML per le tre tipologie di card (ricetta, ricettario, recensione). Nessuna dipendenza dal DOM — restituisce solo stringhe HTML.
- `componenti/azioni-card.js` : event delegation globale per i pulsanti delle card, modale di conferma azioni.
- `componenti/modale-recensione.js` : logica della modale di inserimento/modifica recensione, con upsert e precompilazione.
- `viste/vista-*.js` : un file per ogni rotta. Ciascuno gestisce il ciclo di vita della propria vista: inizializzazione DOM, fetch dati, event listener locali.

10. Checklist di test manuali - tutti passati!

Prima della presentazione, verificare il corretto funzionamento dei seguenti scenari:

Registrazione e login:

- ☐ Registrazione con tutti i campi compila correttamente `users` in localStorage (verificare in DevTools: nessun campo `password`, solo `passwordHash` e `salt`).

- ☐ Accettazione Termini e Privacy: il pulsante "Ho letto, torna al form" si abilita solo dopo aver scrollato tutto il documento.
- ☐ Login con username e con email funzionano entrambi.
- ☐ "Ricordami" scrive `{ username, displayName, encryptedPassword, iv }` in `localStorage["cc_remembered_accounts"]` — nessun campo in chiaro.
- ☐ Refresh della pagina: si rimane loggati (`localStorage session` è persistente).
- ☐ Logout: `session.currentUserId` torna `null`, l'utente è rimosso dalla navbar.

Rotte protette:

- ☐ Aprire `#/ricettario`, `#/recensioni`, `#/profilo` da sloggati reindirige a `#/accesso`.

Ricerca ricette:

- ☐ Ricerca per nome trova ricette in tutto il catalogo (es. "pizza", "sushi").
- ☐ Ricerca per ingrediente (es. "chicken") restituisce risultati.
- ☐ Ricerca per lettera filtra correttamente.
- ☐ "Sfoglia tutto" carica il catalogo completo con indice alfabetico.

Ricettario:

- ☐ Aggiunta ricetta dal dettaglio: compare in `cookbook:<id>` e nel ricettario.
- ☐ Rimozione con conferma modale: la ricetta sparisce da `cookbook:<id>`.
- ☐ Nota privata salvata: compare in `cookbook:<id>.notesByRecipeId`.

Recensioni:

- ☐ Inserimento recensione: compare in `reviews:<idRicetta>` con `difficulty`, `taste`, `cookedAt`.
- ☐ Modifica recensione: aggiorna il record esistente (stesso id).
- ☐ Rimozione recensione: scompare dall'array in `reviews:<idRicetta>`.

Profilo:

- ☐ Modifica dati: richiede conferma password, aggiorna `users`.
- ☐ Cambio password: `passwordHash` e `salt` cambiano in `users`.
- ☐ Eliminazione profilo: rimuove utente da `users`, cancella `cookbook:<id>` e tutte le `reviews:*` associate.

11. Scelte implementative e motivazioni

Questa sezione raccoglie le principali decisioni tecniche, con le motivazioni, come richiesto dalla specifica del docente.

- **SPA + hash routing**: permette navigazione fluida senza server configurato; il routing client-side funziona anche aprendo il file direttamente con Live Server.
 - **Catalogo completo allo startup**: soddisfa il requisito di specifica ("tutti i dati scaricati dalle API e memorizzati nel web storage") e rende la ricerca locale e istantanea. La cache TTL evita ri-download inutili.
 - **Ricerca locale sulla cache**: risultati immediati, funziona offline, nessun carico sull'API pubblica.
 - **Hashing SHA-256 + salt** (Web Crypto): la password non è mai in chiaro nel storage. Il salt rende inefficaci le rainbow table.
 - **Namespacing chiavi** (`cookbook:<id>` , `reviews:<id>`): separa nettamente le entità nel localStorage, semplifica la demo e la pulizia selettiva.
 - **Schema versionato + migrazione**: modifiche future al formato dati non rompono i dati esistenti degli utenti.
 - **Upsert recensione**: un utente ha una sola recensione per ricetta; riaprire la modale la modifica, non la duplica.
 - **Aree da catalogo, non da `list.php?a=list`**: evita un elenco di 195 paesi per lo più senza ricette; ogni paese selezionabile produce risultati reali.
 - **Webfont bandiere self-hosted** (Twemoji Country Flags): risolve il mancato rendering delle flag emoji su Windows, con impatto nullo sulle altre parti del testo grazie a `unicode-range`.
 - **Event delegation globale** per le card: i listener sopravvivono alla ricreazione delle card via innerHTML.
 - **Modali di conferma** per le azioni distruttive: prevengono eliminazioni/rimozioni accidentali.
 - **"Ricordami" con AES-GCM**: la password viene cifrata (chiave = `passwordHash` SHA-256, IV casuale) prima di scrivere in `localStorage["cc_remembered_accounts"]`. In DevTools si vede solo un blob Base64; la decifratura avviene on-the-fly al momento del login. La vecchia `cc_accesso_ricorda` in sessionStorage è stata rimpiazzata da questo schema.
 - **Account picker con typewriter**: al secondo accesso compare una modale con gli account ricordati (avatar con iniziali, nome, username). Selezionandone uno, i campi username e password si compilano con un effetto typewriter (30–50ms per carattere), rendendo il flusso visivamente intuitivo.
 - **mostraModalConfermaGenerica** esportata da `azioni-card.js`: unico punto di ingresso per le modali di conferma in tutta l'app, elimina ogni `window.confirm()` nativo.
 - **Re-render live post-recensione**: dopo il salvataggio di una recensione dalla scheda ricetta, la vista si aggiorna completamente senza reload (la ricetta è in cache → istantaneo), aggiornando pulsanti e lista in tempo reale.
-

12. Guida alla demo nei DevTools (flusso di autenticazione)

Questa sezione è pensata per essere consultata durante la discussione orale. Aprire **F12** → **Application** → **Local Storage** / **Session Storage** e tenere il pannello aperto mentre si eseguono le operazioni seguenti.

1. **Registrazione** (`#/registrazione`): compilare il form e inviare. In `localStorage["users"]` compare il nuovo oggetto utente con `passwordHash` e `salt` — il campo `password` è assente. In `sessionStorage` appare temporaneamente `cc_post_signup` e poi sparisce non appena la pagina di login la legge.
2. **Login** (`#/accesso`): inserire le credenziali. La password viene ri-hashata in locale e confrontata con `passwordHash`. Al successo `localStorage["session"].currentUserId` passa da `null` all'id utente e `loginAt` registra il timestamp ISO. La navbar cambia da "Registrati / Accedi" a "Ciao <nome>".
3. **"Ricordami"**: se la casella è spuntata, la password viene cifrata AES-GCM e salvata in `localStorage["cc_remembered_accounts"]` insieme a username e nome visualizzato. In DevTools si vede il campo `encryptedPassword` come blob Base64 — mai la password in chiaro. Al prossimo accesso compare il picker degli account ricordati.
4. **Refresh (F5)**: `session` è in `localStorage`, persistente tra reload. `inizializzaStorage` è idempotente e non sovrascrive le chiavi esistenti: si rimane loggati senza rifare il login.
4b. **Account picker**: dopo il logout, riaprire la pagina di login — dopo 2 secondi appare la modale con gli account ricordati. Selezionandone uno, i campi username e password si compilano con effetto typewriter. Ispezionare `localStorage["cc_remembered_accounts"]`: si vede `encryptedPassword` (blob Base64), mai la password in chiaro.
5. **Rotte protette**: svuotare `session` dal pannello Application e navigare a `#/ricettario` — il router rileva l'assenza di sessione e reindirige a `#/accesso`.
6. **Aggiunta al ricettario**: cliccare "Aggiungi al ricettario" su una ricetta. In `localStorage["cookbook:<idUser>"]` l'id della ricetta compare in `recipeIds`.
7. **Inserimento recensione**: compilare la modale. In `localStorage["reviews:<idRicetta>"]` appare l'oggetto con `cookedAt`, `difficulty` e `taste`.
8. **Logout**: `session.currentUserId` torna `null`. L'array `users`, il ricettario e le recensioni restano intatti. La home pubblica viene mostrata immediatamente senza reload: se l'utente era già su `#/home` (home loggata), impostare lo stesso hash non emette `hashchange` — viene dispatchiato manualmente un `HashChangeEvent` per forzare il router a re-renderizzare la vista pubblica.

Tutti questi punti sono annotati nel codice con il tag `DEVTOOLS:` e `>>> MOMENTO CHIAVE <<<` per facilitarne la localizzazione durante la sessione.

13. Funzionalità aggiuntive (oltre la specifica)

La specifica consente esplicitamente di implementare funzionalità extra. Le seguenti sono state aggiunte per migliorare l'esperienza utente senza deviare dai requisiti richiesti:

- **Home personalizzata per utenti loggati:** la vista `home.logged.html` mostra fino a 4 ricette casuali per il paese di origine e 4 per il paese di residenza dell'utente, ricavate dalla cache locale filtrando per area. Se i due paesi coincidono, viene mostrata un'unica sezione.
- **Aree/paesi tradotti in italiano con emoji bandiera:** tutte le 37 cucine presenti nel catalogo sono accompagnate dal nome italiano e dalla bandiera del paese, resa correttamente su tutti i sistemi operativi grazie al webfont Twemoji Country Flags.
- **Anteprima e player YouTube inline:** nella scheda dettaglio, se TheMealDB fornisce un link video, viene mostrata l'anteprima del thumbnail; al click si sostituisce con il player embed senza lasciare la pagina.
- **Indice alfabetico interattivo:** nella modalità "Sfoglia tutto" di Esplora, un indice sticky con le lettere A–Z permette di saltare direttamente alla sezione desiderata; la lettera corrente si evidenzia automaticamente durante lo scroll tramite `IntersectionObserver`.
- **Modali legali con lettura obbligatoria e checkbox bloccati:** i documenti "Termini e condizioni" e "Informativa sulla privacy" si aprono in modali dedicate; il pulsante "Ho letto, torna al form" si abilita solo dopo aver scrollato tutto il documento; i relativi checkbox partono `disabled` e si abilitano (auto-spuntandosi) solo alla chiusura confermata — il bypass è fisicamente impossibile.
- **Modale di conferma per azioni distruttive:** aggiunta, rimozione dal ricettario, rimozione recensione e logout passano tutti per una modale di conferma custom — nessun `window.confirm()` nativo nell'app.
- **Feedback di caricamento con spinner tematico:** un overlay a tutto schermo copre la pagina durante il download del catalogo; spinner inline compaiono nelle viste Esplora, Login e Registrazione durante le operazioni asincrone.
- **Navbar sticky:** la barra di navigazione rimane sempre visibile in cima alla pagina durante lo scroll.
- **Account picker "Ricordami" con AES-GCM e typewriter:** al secondo accesso una modale mostra gli account ricordati (avatar con iniziali); selezionandone uno i campi si compilano con effetto typewriter; la password è cifrata AES-256-GCM in `localStorage`, mai in chiaro.
- **Glow highlight su card recensione:** cliccando "Guarda la tua recensione" la SPA naviga a `#/recensioni`, fa autoscroll alla card corrispondente e la fa pulsare con un glow ciano/viola per 3 secondi.
- **Validazione form avanzata:** scroll automatico all'alert di errore, glow rosso animato sui campi invalidi (si auto-rimuove al primo `input`), fix del colore sfondo su campi compilati automaticamente dal browser.

- **Pulsante chiusura unificato:** tutte le "x" dell'app (modali Bootstrap, doc-modali, offcanvas mobile) usano un unico componente CSS (rosso, bordi arrotondati, posizionato `absolute top-right`).
-

14. Limiti noti

È buona pratica dichiarare con onestà i limiti dell'implementazione, anche in un contesto didattico.

- **Persistenza locale al browser:** i dati vivono nel web storage di quel browser e profilo specifico. Non c'è sincronizzazione tra dispositivi diversi, il che è coerente con l'assenza di backend richiesta dalla specifica.
 - **Dipendenza dalla rete al primo avvio:** il primo caricamento richiede una connessione attiva per scaricare il catalogo da TheMealDB. Ai riavvii successivi la cache in `localStorage` permette la navigazione anche offline.
 - **Contenuto in inglese:** nomi, istruzioni e categorie delle ricette provengono direttamente da TheMealDB in inglese e non vengono tradotte (farlo manualmente per oltre 600 ricette non è praticabile). L'app traduce invece le aree geografiche, che sono un insieme finito e gestibile.
 - **Capienza del `localStorage` (~5 MB):** il catalogo completo occupa diversi MB; sui browser moderni il limite è in genere 5–10 MB, sufficiente per questa applicazione, ma è un parametro da tenere a mente se il catalogo TheMealDB crescesse significativamente.
 - **Sicurezza lato client:** in assenza di backend, un utente tecnicamente esperto potrebbe manipolare il `localStorage` direttamente. Questo esula completamente dagli obiettivi e dai vincoli del corso; l'implementazione adottata (hashing SHA-256 + salt) è la più robusta possibile entro i limiti imposti dalla specifica.
-

15. Appendice A — Mappa file-per-file della repository

Questa appendice descrive **ogni file e cartella** del progetto e il suo ruolo nell'ecosistema, in modo che la coesione dell'insieme sia evidente. La struttura è volutamente piatta e leggibile: nessun file è superfluo, e ogni modulo ha una sola responsabilità ben definita.

A.1 Albero del progetto

```
CyberCuisineV0/  
├── .editorconfig           # convenzioni di formattazione cross-editor  
├── .prettierrc / .prettierrcignore # regole e ignore del formatter
```

```

├─ .gitignore          # cosa non versionare (node_modules, note
personali, possibili_domande.md)
├─ LICENSE             # licenza MIT
├─ README.md          # guida rapida + mappa web storage + reset dati
├─ docs/
|   └─ screenshots/    # schermate dimostrative (prove di funzionamento
richieste dalla specifica)
|       └─ core_docs/
|           └─ relazione.md      # questo documento
|           └─ possibili_domande.md # preparazione orale (NON versionato)
|           └─ PWM_ProgettoAnnoAccademico20242025.pdf # specifica del docente
└─ src/
    ├─ html/           # frammenti SPA (struttura, HTML5)
    ├─ css/            # presentazione (CSS3), separata dalla struttura
    ├─ js/             # logica applicativa (ES6 modules)
    └─ assets/         # font ed immagini statiche

```

A.2 src/html/ — struttura delle pagine (HTML5)

Tutti i file sono **frammenti** iniettati dal router nel contenitore `<main id="app">`, tranne `index.html` che è la pagina-guscio.

- `index.html` — la pagina-guscio della SPA: include `<head>` (font, Bootstrap CDN, i 7 CSS), la navbar con menu utente e link auth, l'**overlay di avvio**, il `<main id="app">`, le **modali globali** (conferma azioni `#modalAzioniRicetta`, conferma logout `#modalLogout`), il footer e il caricamento di `main.js` come modulo. È l'unico file HTML realmente aperto dal browser.
- `home.html` — home **pubblica** (utente sloggato): hero + griglia "Ricette in evidenza".
- `home.logged.html` — home **autenticata**: sezioni "Dal tuo paese di origine/residenza" popolate filtrando il catalogo per area dell'utente.
- `login.html` — form di accesso, casella "Ricordami", modale del picker account ricordati, spinner.
- `register.html` — form di registrazione con select paesi, piatti preferiti, modali legali (Termini/Privacy) e checkbox bloccati.
- `profile.html` — gestione profilo: dati personali (campi inizialmente `disabled`), modale di ri-autenticazione password, pulsanti modifica/salva/elimina.
- `esplora.html` — ricerca (select tipo + campo termine), pulsante "Sfoggia l'intero catalogo", barra indice alfabetico, contenitore risultati.
- `ricettario.html` — elenco ricette salvate (rotta protetta).
- `reviews.html` — "Le tue recensioni" (rotta protetta): contenitore `#elencoRecensioni`.
- `recipe-detail.html` — scheda della singola ricetta: contenitore `#dettaglioRicetta` e sezione recensioni.

A.3 src/css/ — presentazione (CSS3)

Caricati tutti da `index.html`, nell'ordine: `theme` → `base` → `layout` → `components` → `forms` → `utilities` → `modal`. La separazione tematica rende manutenibile lo stile e materializza il requisito di **separazione struttura/presentazione**.

- `theme.css` — variabili CSS globali (design token): palette cyberpunk (viola `#b312ff`, ciano `#00f0ff`, sfondo `#0c0b1a`). Cambiando qui i token cambia tutta l'app.
- `base.css` — reset minimo, `@font-face` Twemoji Country Flags (limitato via `unicode-range` alle bandiere), font-family del body, stili base di titoli e link.
- `layout.css` — header sticky, navbar, footer, hero, impaginazione della vista Esplora.
- `components.css` — card con effetto glow, pulsanti tematici, indice alfabetico, elementi del ricettario, highlight delle recensioni.
- `forms.css` — input, select, placeholder, testi d'aiuto, modali legali, spinner, fix autofill del browser, glow sui campi invalidi.
- `modal.css` — stile delle modali Bootstrap personalizzate e pulsante di chiusura unificato.
- `utilities.css` — classi helper varie (spaziature, colori accento, ecc.).

A.4 src/js/ — logica (ES6 modules)

Per le responsabilità dettagliate dei moduli si veda §9.2. In sintesi, file per file:

- `main.js` — punto di ingresso: orchestrazione dello startup (`init storage` → `navbar` → `event delegation` → `download catalogo` → `router`).
- `costanti.js` — chiavi del web storage, chiavi legacy, `SCHEMA_VERSION`, `BASE_API`.
- `stato.js` — stato volatile in-memory (cache frammenti, risultati ricerca, hash attivo, highlight recensione).
- `storage.js` — tutta la persistenza: helper `localStorage`, `init+migrazione schema`, `CRUD utenti/sessione/ricettario/recensioni`, `generazione id`.
- `auth.js` — crittografia: salt, hashing SHA-256, verifica password (Web Crypto). Nessuna dipendenza da DOM o storage.
- `ui.js` — utility UI riutilizzabili: `mostraAvviso` (alert con autoscroll), `generaId`, `evidenziaFieldInvalidi`.
- `navbar.js` — aspetto della navbar in base alla sessione, `logout` con modale, `evidenziazione link attivo`.
- `rotte.js` — mappa dichiarativa `hash` → { frammento, handler, protetta }.
- `router.js` — `gestisciCambioRoute`: normalizza hash, controlla protezione, carica frammento (con cache), invoca l'handler di vista.
- `gestione-api/api.js` — comunicazione TheMealDB, normalizzazione ricette, cache catalogo (TTL 72h), aree con emoji/traduzioni.

- `componenti/carte.js` — generatori di markup per le card (ricetta, ricettario, recensione). Restituiscono solo stringhe HTML.
- `componenti/azioni-card.js` — event delegation globale dei pulsanti card + modale di conferma riutilizzabile (`mostraModalConfermaGenerica`).
- `componenti/modale-recensione.js` — modale inserimento/modifica recensione, con upsert e precompilazione.
- `viste/vista-*.js` — un controller per ogni rotta: `vista-home` , `vista-home-loggata` , `vista-accesso` , `vista-registrazione` , `vista-profilo` , `vista-esplora` , `vista-ricettario` , `vista-recensioni` , `vista-dettaglio-ricetta` .

A.5 `src/assets/` — risorse statiche

- `fonts/TwemojiCountryFlags.woff2` — webfont self-hosted per rendere le bandiere emoji su tutti i sistemi (incluso Windows).
- `img/cc-logo.png` — logo del marchio nella navbar.
- `img/piadina-spinner.png` — immagine dello spinner tematico (overlay di avvio e attese).
- `img/.gitkeep` / `json/.gitkeep` — file segnaposto per versionare cartelle altrimenti vuote (la `json/` è predisposta per eventuali dati statici futuri).

A.6 Coesione dell'insieme

Il flusso end-to-end mostra come i pezzi si incastrano: `main.js` avvia lo storage (`storage.js`) e scarica il catalogo (`api.js`), poi attiva il router (`router.js` + `rotte.js`); ogni navigazione carica un frammento da `src/html/` e invoca la `vista-*.js` corrispondente, che legge/scrive il dominio tramite `storage.js` , genera markup con `componenti/carte.js` e reagisce ai click via `componenti/azioni-card.js` ; la presentazione è interamente delegata a `src/css/` , e le credenziali passano sempre da `auth.js` . Nessun modulo conosce più del necessario: è questa separazione netta delle responsabilità a rendere il progetto manutenibile ed estendibile (aggiungere una vista = un frammento HTML + un `vista-*.js` + una riga in `rotte.js`).

16. Conclusioni

CyberCuisine è una dimostrazione pratica di come sia possibile realizzare un'applicazione web completa e funzionale usando esclusivamente le tecnologie native della piattaforma web, senza framework, senza build step e senza backend. L'architettura SPA con hash routing, la persistenza strutturata nel web storage, l'integrazione con un'API REST esterna e la gestione sicura delle credenziali mostrano una comprensione concreta e applicata dei principi fondamentali dello sviluppo web moderno.

Il codice è stato pensato per essere leggibile e auto-documentante: ogni modulo ha responsabilità chiare e delimitate, i commenti inline spiegano il "perché" delle scelte non ovvie, e i punti chiave per la demo — le scritture nel web storage durante login, logout, registrazione, aggiunta al ricettario e inserimento recensioni — sono segnalati con tag `DEVT00LS:` che rendono immediato orientarsi nel pannello Application dei DevTools del browser.